

Student Performance Prediction

Practical AI/ML Week 1 Project

4th-Year Computer Science Student

AI/ML Specialization

September 28, 2026

Abstract

This document presents an end-to-end Machine Learning project for Student Performance Prediction. The project demonstrates practical Python programming, data loading, data cleaning, exploratory data analysis, feature engineering, pre-processing, model training, model evaluation, prediction, visualization, and model serialization.

The implementation uses Python, Pandas, NumPy, Matplotlib, and Scikit-learn. Two regression models are trained and compared: Linear Regression and Random Forest Regression.

Contents

1 Project Overview	5
2 Development Environment	5
3 Dataset	6
4 Target Variable	6
5 Target Leakage Prevention	6
6 Machine Learning Inputs and Target	7
7 Data Preprocessing	7
8 Training and Testing	8

9 Machine Learning Models	8
9.1 Linear Regression	9
9.2 Random Forest Regression	9
10 Model Evaluation	9
10.1 Mean Absolute Error	10
10.2 Mean Squared Error	10
10.3 Root Mean Squared Error	10
10.4 R-squared	10
11 Exploratory Data Analysis	10
12 Visualizations	11
13 Feature Importance	12
14 Model Serialization	12
15 Generated Outputs	12
16 Project Limitations	13
17 Professional ML Workflow	13
18 Complete Python Implementation	15
19 Code Explanation by Major Section	39
19.1 Imports	39
19.2 Paths	39
19.3 Dataset Loading	39
19.4 Validation	39
19.5 Cleaning	40
19.6 Feature Engineering	40
19.7 Exploratory Data Analysis	40
19.8 Preprocessing	40
19.9 Training	40
19.10 Evaluation	41
19.11 Model Selection	41
19.12 Prediction	41
19.13 Serialization	41
20 Possible Interview Questions	41

21 Future Improvements	42
-------------------------------	-----------

22 Conclusion	43
----------------------	-----------

1 Project Overview

The objective of this project is to build a practical Machine Learning solution that predicts student academic performance from the information available in the dataset.

The project follows the following pipeline:

Data → Data Cleaning → Exploratory Data Analysis → Feature Engineering → Preprocessing → Model Training → Evaluation → Prediction

The project is intended to demonstrate the following skills:

- Python programming
- Pandas data manipulation
- NumPy numerical operations
- Data visualization using Matplotlib
- Data cleaning
- Feature engineering
- Categorical feature encoding
- Machine Learning pipelines
- Regression modeling
- Model evaluation
- Model serialization

2 Development Environment

The project uses the following technologies:

Technology	Purpose
Python	Main programming language
VS Code	Development environment
Jupyter Notebook	Data exploration and experimentation
GitHub	Version control and portfolio management
Pandas	Data manipulation
NumPy	Numerical operations
Matplotlib	Data visualization
Scikit-learn	Machine Learning
Joblib	Saving the trained model

Table 1: Technology Stack

3 Dataset

The dataset is stored in:

`data/exams.csv`

The dataset contains the following columns:

Column	Description
gender	Student gender
race/ethnicity	Student demographic group
parental level of education	Parent/guardian education level
lunch	Lunch category
test preparation course	Test preparation status
math score	Mathematics examination score
reading score	Reading examination score
writing score	Writing examination score

Table 2: Dataset Features

4 Target Variable

The project creates a new target variable called **overall_score**.

It is calculated using:

$$\text{overall_score} = \frac{\text{math score} + \text{reading score} + \text{writing score}}{3}$$

For example, if a student has:

$$\text{Math} = 60, \quad \text{Reading} = 70, \quad \text{Writing} = 80$$

then:

$$\text{overall_score} = \frac{60 + 70 + 80}{3} = 70$$

5 Target Leakage Prevention

An important Machine Learning consideration in this project is **target leakage**.

The target is calculated directly from the following fields:

- math score
- reading score

- writing score

Therefore, these three variables are not used as model input features.

The model instead uses:

- gender
- race/ethnicity
- parental level of education
- lunch
- test preparation course

Using the subject scores as input variables would provide the model with information that directly determines the target and would produce an unrealistic evaluation.

6 Machine Learning Inputs and Target

The feature matrix is:

$$X = [\text{gender}, \text{race/ethnicity}, \text{parental education}, \text{lunch}, \text{test preparation}]$$

The target vector is:

$$y = \text{overall_score}$$

In Machine Learning terminology:

- X represents the independent variables or features.
- y represents the dependent variable or target.

7 Data Preprocessing

The categorical features cannot be passed directly into the selected Machine Learning algorithms as strings.

Therefore, the project uses:

1. **SimpleImputer** to handle missing categorical values.
2. **OneHotEncoder** to convert categorical values into numerical representations.
3. **ColumnTransformer** to apply preprocessing to the appropriate columns.

4. **Pipeline** to combine preprocessing and Machine Learning into a reproducible workflow.

For example, a categorical value such as:

```
gender = female
```

can be represented internally using numerical indicator variables.

The encoder is configured with:

```
handle_unknown="ignore"
```

so that previously unseen categories during prediction do not cause the transformation process to fail.

8 Training and Testing

The dataset is divided into two groups:

- 80% training data
- 20% testing data

The project uses:

```
test_size=0.20
```

and:

```
random_state=42
```

The fixed random state ensures that the data split is reproducible.

The training set is used to learn model parameters, while the test set is reserved for evaluating performance on unseen examples.

9 Machine Learning Models

Two regression models are implemented.

9.1 Linear Regression

Linear Regression is used as the baseline model.

The goal of the baseline is to establish a simple reference point before evaluating a more flexible model.

Conceptually, a linear regression model attempts to learn a relationship of the form:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

where:

- $\hat{y}$ is the predicted score,
- β_0 is the intercept,
- β_i are learned coefficients,
- x_i are input features.

9.2 Random Forest Regression

The project also uses a Random Forest Regressor.

A Random Forest is an ensemble of decision trees. The implementation uses:

```
n_estimators=300
```

which means the forest contains 300 trees.

Additional parameters are used to control tree structure:

```
min_samples_split=5
```

```
min_samples_leaf=2
```

The model uses:

```
n_jobs=-1
```

to make use of available CPU cores.

10 Model Evaluation

The project evaluates models using three metrics.

10.1 Mean Absolute Error

Mean Absolute Error is:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

It represents the average absolute prediction error.

Lower values indicate smaller average errors.

10.2 Mean Squared Error

Mean Squared Error is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The project uses the square root of MSE to obtain RMSE.

10.3 Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE is expressed in the same units as the prediction target.

10.4 R-squared

The project also calculates:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

R-squared measures how well the model explains variation in the target variable.

11 Exploratory Data Analysis

The project performs multiple exploratory analyses.

These include:

- Dataset dimensions
- Data types
- Missing values
- Duplicate rows

- Score ranges
- Summary statistics
- Category distributions
- Average scores
- Grouped performance analysis

The following grouped analyses are included:

- Overall score by test preparation course
- Overall score by gender
- Overall score by parental education
- Overall score by lunch type

12 Visualizations

The project generates the following visualizations:

1. Distribution of overall student scores
2. Average score by subject
3. Average overall score by test preparation
4. Average overall score by gender
5. Average overall score by parental education
6. Average overall score by lunch type
7. Actual versus predicted scores

The graphs are saved in the **outputs** directory.

13 Feature Importance

The Random Forest model provides feature importance information.

Because categorical variables are converted into multiple one-hot encoded columns, the resulting feature names represent the encoded categories.

Feature importance can be used to understand which transformed inputs were useful to the Random Forest model.

However, feature importance should not automatically be interpreted as causal influence.

For example, a feature having high importance means that it helped the model make predictions; it does not by itself prove that the feature caused the outcome.

14 Model Serialization

The trained best model is saved using Joblib.

The resulting file is:

```
outputs/student_performance_model.pkl
```

This allows the trained pipeline to be reused later without retraining.

A saved model can later be loaded using:

```
model = joblib.load("student_performance_model.pkl")
```

and predictions can be generated using:

```
model.predict(new_data)
```

15 Generated Outputs

The program generates the following files:

File	Purpose
cleaned_student_data.csv	Cleaned dataset
model_comparison.csv	Comparison of the trained models
test_predictions.csv	Actual and predicted test values
feature_importance.csv	Random Forest feature importance
student_performance_model.pkl	Serialized trained model
project_summary.txt	Project summary
overall_score_distribution.png	Overall score histogram

average_subject_scores.png	Average subject score chart
score_by_test_preparation.png	Preparation analysis chart
score_by_gender.png	Gender analysis chart
score_by_parental_education.png	Education analysis chart
score_by_lunch.png	Lunch analysis chart
actual_vs_predicted.png	Model prediction comparison chart

16 Project Limitations

The dataset does not contain variables such as:

- Study hours
- Attendance
- Previous grades
- Longitudinal academic history

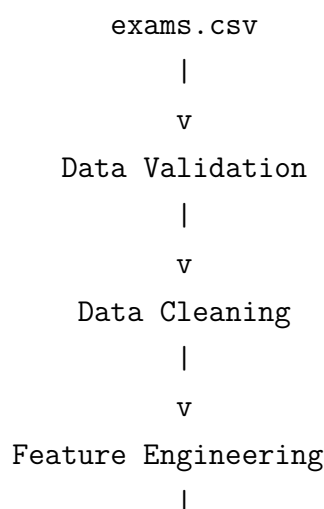
Therefore, the model cannot evaluate the effect or predictive value of those factors.

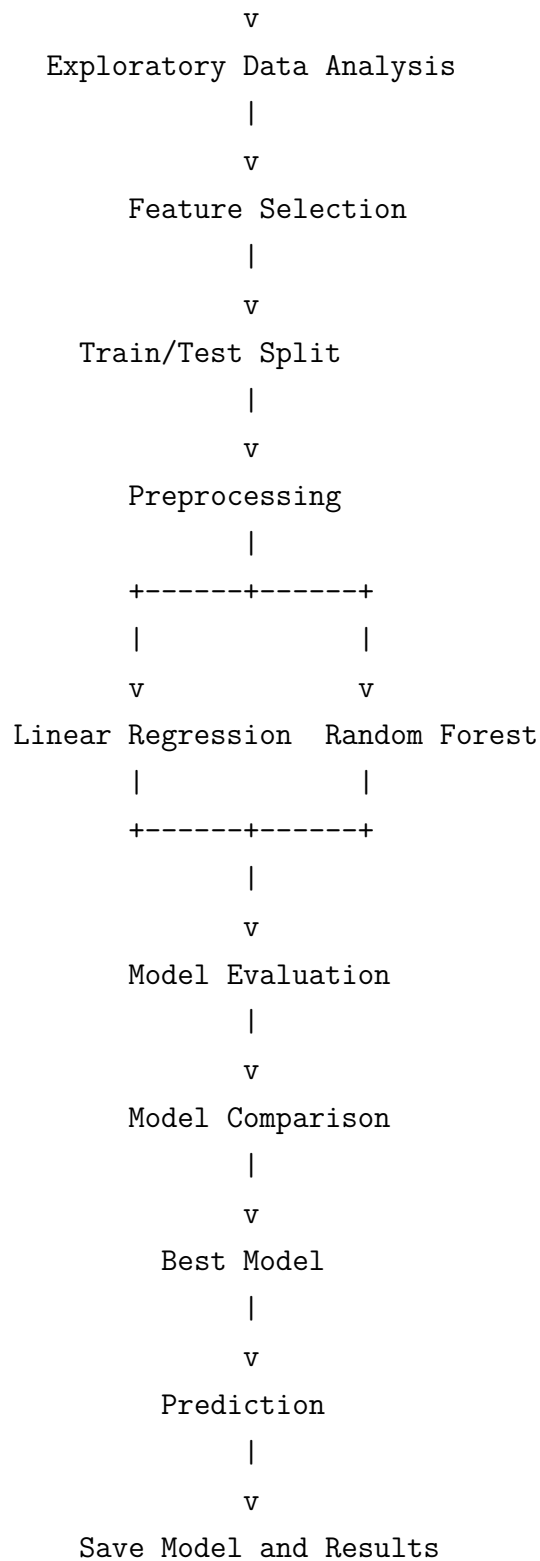
The current project should be viewed as a practical Machine Learning learning project rather than a production-ready student-risk system.

Another limitation is that the target is constructed from the three exam scores. Therefore, the system estimates the overall exam result from the available student attributes rather than predicting a future exam before those scores exist.

17 Professional ML Workflow

The complete implementation follows this conceptual architecture:





18 Complete Python Implementation

The following listing contains the complete Python implementation of the Student Performance Prediction project.

```
1
2 """
3 Student Performance Prediction
4 Week 1 - Practical AI/ML Learning Program
5
6 Project Flow:
7 Data
8 -> Data Cleaning
9 -> Exploratory Data Analysis
10 -> Feature Engineering
11 -> Train/Test Split
12 -> Baseline Model
13 -> Improved Model
14 -> Evaluation
15 -> Prediction
16 -> Save Results
17
18 Dataset:
19     data/exams.csv
20
21 Target:
22     overall_score
23
24 Important:
25 The dataset contains math, reading, and writing scores.
26 Because overall_score is calculated from these three scores,
27 those three columns are NOT used as input features.
28 This prevents target leakage.
29 """
30
31 from pathlib import Path
32
33 import numpy as np
34 import pandas as pd
35 import matplotlib.pyplot as plt
36
37 from sklearn.compose import ColumnTransformer
38 from sklearn.ensemble import RandomForestRegressor
39 from sklearn.impute import SimpleImputer
40 from sklearn.linear_model import LinearRegression
41 from sklearn.metrics import (
42     mean_absolute_error,
```

```
43     mean_squared_error ,
44     r2_score ,
45 )
46 from sklearn.model_selection import train_test_split
47 from sklearn.pipeline import Pipeline
48 from sklearn.preprocessing import OneHotEncoder
49
50 import joblib
51
52
53 # =====
54 # 1. PATHS
55 # =====
56
57 PROJECT_ROOT = Path(__file__).resolve().parent
58 DATA_PATH = PROJECT_ROOT / "data" / "exams.csv"
59 OUTPUT_DIR = PROJECT_ROOT / "outputs"
60
61 OUTPUT_DIR.mkdir(exist_ok=True)
62
63 print("=" * 80)
64 print("STUDENT PERFORMANCE PREDICTION PROJECT")
65 print("=" * 80)
66
67
68 # =====
69 # 2. CHECK DATASET
70 # =====
71
72 print("\n[1] Checking dataset...")
73
74 if not DATA_PATH.exists():
75     raise FileNotFoundError(
76         f"""
77 Dataset not found.
78
79 Expected location:
80 {DATA_PATH}
81
82 Make sure your file is named:
83 exams.csv
84
85 and is inside:
86 data/
87 """
88     )
89
```

```
90 print(f"Dataset found: {DATA_PATH}")
91
92
93 # =====
94 # 3. LOAD DATASET
95 # =====
96
97 print("\n[2] Loading dataset...")
98
99 df = pd.read_csv(DATA_PATH)
100
101 print(f"Rows      : {df.shape[0]}")
102 print(f"Columns   : {df.shape[1]}")
103
104 print("\nFirst 5 rows:")
105 print(df.head())
106
107
108 # =====
109 # 4. CHECK EXPECTED COLUMNS
110 # =====
111
112 print("\n[3] Validating columns...")
113
114 expected_columns = [
115     "gender",
116     "race/ethnicity",
117     "parental level of education",
118     "lunch",
119     "test preparation course",
120     "math score",
121     "reading score",
122     "writing score",
123 ]
124
125 missing_columns = [
126     column
127     for column in expected_columns
128     if column not in df.columns
129 ]
130
131 if missing_columns:
132     raise ValueError(
133         "The following required columns are missing:\n"
134         + "\n".join(f"- {column}" for column in missing_columns)
135     )
136
```

```
137 print("All expected columns are available.")
138
139
140 # =====
141 # 5. DATA TYPES
142 # =====
143
144 print("\n[4] Checking data types...")
145
146 score_columns = [
147     "math score",
148     "reading score",
149     "writing score",
150 ]
151
152 for column in score_columns:
153     df[column] = pd.to_numeric(
154         df[column],
155         errors="coerce"
156     )
157
158 print(df.dtypes)
159
160
161 # =====
162 # 6. DATA QUALITY CHECK
163 # =====
164
165 print("\n[5] Data quality analysis...")
166
167 print("\nMissing values:")
168 print(df.isnull().sum())
169
170 duplicate_count = df.duplicated().sum()
171
172 print(f"\nDuplicate rows: {duplicate_count}")
173
174 print("\nScore ranges:")
175
176 for column in score_columns:
177     print(
178         f"{column}: "
179         f"min={df[column].min()}, "
180         f"max={df[column].max()}"
181     )
182
183
```

```
184 # =====
185 # 7. CLEAN DATA
186 # =====
187
188 print("\n[6] Cleaning data...")
189
190 if duplicate_count > 0:
191     df = df.drop_duplicates().reset_index(drop=True)
192     print(f"Removed {duplicate_count} duplicate rows.")
193
194 # Replace invalid scores with NaN
195 for column in score_columns:
196
197     invalid_values = (
198         (df[column] < 0)
199         | (df[column] > 100)
200     )
201
202     invalid_count = invalid_values.sum()
203
204     if invalid_count > 0:
205         print(
206             f"Replacing {invalid_count} invalid values "
207             f"in {column} with NaN."
208         )
209
210         df.loc[invalid_values, column] = np.nan
211
212 # Remove rows where all three subject scores are missing
213 df = df.dropna(
214     subset=score_columns,
215     how="all"
216 ).reset_index(drop=True)
217
218 print(f"Rows after cleaning: {len(df)}")
219
220
221 # =====
222 # 8. FEATURE ENGINEERING
223 # =====
224
225 print("\n[7] Feature engineering...")
226
227 # Calculate overall score
228 df["overall_score"] = df[score_columns].mean(axis=1)
229
230 print("\nOverall score statistics:")
```

```
231 print(df["overall_score"].describe().round(2))
232
233
234 # Performance category is for interpretation only.
235 def performance_category(score):
236     if pd.isna(score):
237         return "Unknown"
238
239     if score >= 70:
240         return "High"
241
242     if score >= 50:
243         return "Medium"
244
245     return "Low"
246
247
248 df["performance_category"] = (
249     df["overall_score"]
250     .apply(performance_category)
251 )
252
253 print("\nPerformance category counts:")
254 print(df["performance_category"].value_counts())
255
256
257 # =====
258 # 9. SAVE CLEANED DATA
259 # =====
260
261 cleaned_data_path = (
262     OUTPUT_DIR / "cleaned_student_data.csv"
263 )
264
265 df.to_csv(
266     cleaned_data_path,
267     index=False
268 )
269
270 print(
271     f"\nCleaned dataset saved to:\n"
272     f"{cleaned_data_path}"
273 )
274
275
276 # =====
277 # 10. EXPLORATORY DATA ANALYSIS
```

```
278 # =====
279
280 print("\n[8] Exploratory Data Analysis...")
281
282 print("\nDataset information:")
283 print(df.info())
284
285 print("\nSummary statistics:")
286 print(df.describe().round(2))
287
288 categorical_columns = [
289     "gender",
290     "race/ethnicity",
291     "parental level of education",
292     "lunch",
293     "test preparation course",
294 ]
295
296 print("\nCategorical column distributions:")
297
298 for column in categorical_columns:
299     print(f"\n{column}")
300     print(df[column].value_counts())
301
302
303 # =====
304 # 11. EDA OBSERVATIONS
305 # =====
306
307 print("\n" + "=" * 80)
308 print("EDA ANALYSIS")
309 print("=" * 80)
310
311 overall_mean = df["overall_score"].mean()
312 overall_median = df["overall_score"].median()
313 highest_score = df["overall_score"].max()
314 lowest_score = df["overall_score"].min()
315
316 print(f"\nAverage overall score : {overall_mean:.2f}")
317 print(f"Median overall score : {overall_median:.2f}")
318 print(f"Highest overall score : {highest_score:.2f}")
319 print(f"Lowest overall score : {lowest_score:.2f}")
320
321
322 # Test preparation analysis
323 preparation_analysis = (
324     df.groupby("test preparation course")
```

```
325     ["overall_score"]
326     .agg(["count", "mean", "median"])
327     .round(2)
328 )
329
330 print("\nOverall score by test preparation:")
331 print(preparation_analysis)
332
333
334 # Gender analysis
335 gender_analysis = (
336     df.groupby("gender")
337     ["overall_score"]
338     .agg(["count", "mean", "median"])
339     .round(2)
340 )
341
342 print("\nOverall score by gender:")
343 print(gender_analysis)
344
345
346 # Parental education analysis
347 education_analysis = (
348     df.groupby("parental level of education")
349     ["overall_score"]
350     .agg(["count", "mean", "median"])
351     .sort_values("mean", ascending=False)
352     .round(2)
353 )
354
355 print("\nOverall score by parental education:")
356 print(education_analysis)
357
358
359 # Lunch analysis
360 lunch_analysis = (
361     df.groupby("lunch")
362     ["overall_score"]
363     .agg(["count", "mean", "median"])
364     .round(2)
365 )
366
367 print("\nOverall score by lunch category:")
368 print(lunch_analysis)
369
370
371 # =====
```

```
372 # 12. VISUALIZATIONS
373 # =====
374
375 print("\nCreating visualizations...")
376
377
378 # Overall score distribution
379 plt.figure(figsize=(9, 6))
380
381 plt.hist(
382     df["overall_score"],
383     bins=20,
384     edgecolor="black"
385 )
386
387 plt.title("Distribution of Overall Student Scores")
388 plt.xlabel("Overall Score")
389 plt.ylabel("Number of Students")
390
391 plt.tight_layout()
392
393 plt.savefig(
394     OUTPUT_DIR / "overall_score_distribution.png",
395     dpi=300
396 )
397
398 plt.show()
399 plt.close()
400
401
402 # Average subject scores
403 subject_means = df[
404     score_columns
405 ].mean()
406
407 plt.figure(figsize=(9, 6))
408
409 plt.bar(
410     subject_means.index,
411     subject_means.values
412 )
413
414 plt.title("Average Score by Subject")
415 plt.xlabel("Subject")
416 plt.ylabel("Average Score")
417
418 plt.tight_layout()
```

```
419
420 plt.savefig(
421     OUTPUT_DIR / "average_subject_scores.png",
422     dpi=300
423 )
424
425 plt.show()
426 plt.close()
427
428
429 # Test preparation
430 plt.figure(figsize=(9, 6))
431
432 preparation_means = (
433     df.groupby("test preparation course")
434     ["overall_score"]
435     .mean()
436 )
437
438 plt.bar(
439     preparation_means.index,
440     preparation_means.values
441 )
442
443 plt.title("Average Overall Score by Test Preparation")
444 plt.xlabel("Test Preparation Course")
445 plt.ylabel("Average Overall Score")
446
447 plt.tight_layout()
448
449 plt.savefig(
450     OUTPUT_DIR / "score_by_test_preparation.png",
451     dpi=300
452 )
453
454 plt.show()
455 plt.close()
456
457
458 # Gender
459 plt.figure(figsize=(9, 6))
460
461 gender_means = (
462     df.groupby("gender")
463     ["overall_score"]
464     .mean()
465 )
```

```
466
467 plt.bar(
468     gender_means.index,
469     gender_means.values
470 )
471
472 plt.title("Average Overall Score by Gender")
473 plt.xlabel("Gender")
474 plt.ylabel("Average Overall Score")
475
476 plt.tight_layout()
477
478 plt.savefig(
479     OUTPUT_DIR / "score_by_gender.png",
480     dpi=300
481 )
482
483 plt.show()
484 plt.close()
485
486
487 # Parental education
488 plt.figure(figsize=(11, 6))
489
490 education_means = (
491     df.groupby("parental level of education")
492     ["overall_score"]
493     .mean()
494     .sort_values(ascending=False)
495 )
496
497 plt.bar(
498     education_means.index,
499     education_means.values
500 )
501
502 plt.title(
503     "Average Overall Score by Parental Level of Education"
504 )
505
506 plt.xlabel("Parental Level of Education")
507 plt.ylabel("Average Overall Score")
508
509 plt.xticks(
510     rotation=45,
511     ha="right"
512 )
```

```
513
514 plt.tight_layout()
515
516 plt.savefig(
517     OUTPUT_DIR / "score_by_parental_education.png",
518     dpi=300
519 )
520
521 plt.show()
522 plt.close()
523
524
525 # Lunch
526 plt.figure(figsize=(9, 6))
527
528 lunch_means = (
529     df.groupby("lunch")
530     ["overall_score"]
531     .mean()
532 )
533
534 plt.bar(
535     lunch_means.index,
536     lunch_means.values
537 )
538
539 plt.title("Average Overall Score by Lunch Type")
540 plt.xlabel("Lunch")
541 plt.ylabel("Average Overall Score")
542
543 plt.tight_layout()
544
545 plt.savefig(
546     OUTPUT_DIR / "score_by_lunch.png",
547     dpi=300
548 )
549
550 plt.show()
551 plt.close()
552
553
554 # =====
555 # 13. PREPARE ML DATA
556 # =====
557
558 print("\n[9] Preparing machine learning data...")
559
```

```
560 # Only use information that could reasonably be considered
561 # input/student-profile information.
562 #
563 # DO NOT use math, reading, or writing score as features because
564 # they are used to calculate overall_score.
565
566 feature_columns = [
567     "gender",
568     "race/ethnicity",
569     "parental level of education",
570     "lunch",
571     "test preparation course",
572 ]
573
574 target_column = "overall_score"
575
576 X = df[feature_columns].copy()
577 y = df[target_column].copy()
578
579 print("\nFeatures:")
580 for feature in feature_columns:
581     print(f"- {feature}")
582
583 print(f"\nTarget: {target_column}")
584
585
586 # =====
587 # 14. TRAIN / TEST SPLIT
588 # =====
589
590 X_train, X_test, y_train, y_test = train_test_split(
591     X,
592     y,
593     test_size=0.20,
594     random_state=42
595 )
596
597 print("\nTraining samples:", len(X_train))
598 print("Testing samples :", len(X_test))
599
600
601 # =====
602 # 15. PREPROCESSING
603 # =====
604
605 categorical_pipeline = Pipeline(
606     steps=[
```

```
607         (
608             "imputer",
609             SimpleImputer(
610                 strategy="most_frequent"
611             )
612         ),
613         (
614             "encoder",
615             OneHotEncoder(
616                 handle_unknown="ignore"
617             )
618         ),
619     ]
620 )
621
622 preprocessor = ColumnTransformer(
623     transformers=[
624         (
625             "categorical",
626             categorical_pipeline,
627             feature_columns
628         )
629     ]
630 )
631
632
633 # =====
634 # 16. BASELINE MODEL      LINEAR REGRESSION
635 # =====
636
637 print("\nTraining Linear Regression baseline...")
638
639 linear_regression_pipeline = Pipeline(
640     steps=[
641         (
642             "preprocessor",
643             preprocessor
644         ),
645         (
646             "model",
647             LinearRegression()
648         )
649     ]
650 )
651
652 linear_regression_pipeline.fit(
653     X_train,
```

```
654     y_train
655 )
656
657 linear_predictions = (
658     linear_regression_pipeline
659     .predict(X_test)
660 )
661
662
663 # =====
664 # 17. RANDOM FOREST MODEL
665 # =====
666
667 print("Training Random Forest model...")
668
669 random_forest_pipeline = Pipeline(
670     steps=[
671         (
672             "preprocessor",
673             preprocessor
674         ),
675         (
676             "model",
677             RandomForestRegressor(
678                 n_estimators=300,
679                 min_samples_split=5,
680                 min_samples_leaf=2,
681                 random_state=42,
682                 n_jobs=-1
683             )
684         )
685     ]
686 )
687
688 random_forest_pipeline.fit(
689     X_train,
690     y_train
691 )
692
693 random_forest_predictions = (
694     random_forest_pipeline
695     .predict(X_test)
696 )
697
698
699 # =====
700 # 18. EVALUATION FUNCTION
```

```
701 # =====
702
703 def evaluate_model(
704     model_name,
705     y_true,
706     predictions
707 ):
708     mae = mean_absolute_error(
709         y_true,
710         predictions
711     )
712
713     rmse = np.sqrt(
714         mean_squared_error(
715             y_true,
716             predictions
717         )
718     )
719
720     r2 = r2_score(
721         y_true,
722         predictions
723     )
724
725     return {
726         "Model": model_name,
727         "MAE": mae,
728         "RMSE": rmse,
729         "R2": r2,
730     }
731
732
733 # =====
734 # 19. EVALUATE MODELS
735 # =====
736
737 print("\n[10] Evaluating models...")
738
739 linear_results = evaluate_model(
740     "Linear Regression",
741     y_test,
742     linear_predictions
743 )
744
745 random_forest_results = evaluate_model(
746     "Random Forest",
747     y_test,
```

```
748     random_forest_predictions
749 )
750
751 results = pd.DataFrame(
752     [
753         linear_results,
754         random_forest_results,
755     ]
756 )
757
758 print("\nModel comparison:")
759 print(
760     results.round(4).to_string(index=False)
761 )
762
763 results.to_csv(
764     OUTPUT_DIR / "model_comparison.csv",
765     index=False
766 )
767
768
769 # =====
770 # 20. SELECT MODEL
771 # =====
772
773 best_index = (
774     results["RMSE"]
775     .idxmin()
776 )
777
778 best_model_name = results.loc[
779     best_index,
780     "Model"
781 ]
782
783 if best_model_name == "Linear Regression":
784
785     best_model = linear_regression_pipeline
786     final_predictions = linear_predictions
787
788 else:
789
790     best_model = random_forest_pipeline
791     final_predictions = random_forest_predictions
792
793
794 print(
```

```
795     f"\nSelected model based on lowest RMSE: "  
796     f"{best_model_name}"  
797 )  
798  
799  
800 # =====  
801 # 21. TEST PREDICTIONS  
802 # =====  
803  
804 prediction_results = pd.DataFrame(  
805     {  
806         "Actual Score": y_test.values,  
807         "Predicted Score": final_predictions,  
808     }  
809 )  
810  
811 prediction_results["Absolute Error"] = (  
812     prediction_results["Actual Score"]  
813     - prediction_results["Predicted Score"]  
814 ).abs()  
815  
816 print("\nSample predictions:")  
817 print(  
818     prediction_results  
819     .head(10)  
820     .round(2)  
821     .to_string(index=False)  
822 )  
823  
824 prediction_results.to_csv(  
825     OUTPUT_DIR / "test_predictions.csv",  
826     index=False  
827 )  
828  
829  
830 # =====  
831 # 22. ACTUAL VS PREDICTED VISUALIZATION  
832 # =====  
833  
834 plt.figure(figsize=(8, 6))  
835  
836 plt.scatter(  
837     y_test,  
838     final_predictions,  
839     alpha=0.6  
840 )  
841
```

```
842 min_score = min(  
843     y_test.min(),  
844     final_predictions.min()  
845 )  
846  
847 max_score = max(  
848     y_test.max(),  
849     final_predictions.max()  
850 )  
851  
852 plt.plot(  
853     [min_score, max_score],  
854     [min_score, max_score],  
855     linestyle="--"  
856 )  
857  
858 plt.title(  
859     f"Actual vs Predicted Overall Scores\n"  
860     f"{best_model_name}"  
861 )  
862  
863 plt.xlabel("Actual Score")  
864 plt.ylabel("Predicted Score")  
865  
866 plt.tight_layout()  
867  
868 plt.savefig(  
869     OUTPUT_DIR / "actual_vs_predicted.png",  
870     dpi=300  
871 )  
872  
873 plt.show()  
874 plt.close()  
875  
876  
877 # =====  
878 # 23. RANDOM FOREST FEATURE IMPORTANCE  
879 # =====  
880  
881 print("\nCalculating feature importance...")  
882  
883 try:  
884  
885     trained_preprocessor = (  
886         random_forest_pipeline  
887         .named_steps["preprocessor"]  
888     )
```

```
889
890     trained_random_forest = (
891         random_forest_pipeline
892         .named_steps["model"]
893     )
894
895     encoded_feature_names = (
896         trained_preprocessor
897         .get_feature_names_out()
898     )
899
900     feature_importances = (
901         trained_random_forest
902         .feature_importances_
903     )
904
905     feature_importance_df = pd.DataFrame(
906         {
907             "Feature": encoded_feature_names,
908             "Importance": feature_importances,
909         }
910     ).sort_values(
911         by="Importance",
912         ascending=False
913     )
914
915     print("\nTop features:")
916     print(
917         feature_importance_df
918         .head(15)
919         .round(4)
920         .to_string(index=False)
921     )
922
923     feature_importance_df.to_csv(
924         OUTPUT_DIR / "feature_importance.csv",
925         index=False
926     )
927
928 except Exception as error:
929
930     print(
931         "Feature importance calculation failed:"
932     )
933
934     print(error)
935
```

```
936
937 # =====
938 # 24. SAVE TRAINED MODEL
939 # =====
940
941 model_path = (
942     OUTPUT_DIR / "student_performance_model.pkl"
943 )
944
945 joblib.dump(
946     best_model,
947     model_path
948 )
949
950 print(
951     f"\nTrained model saved to:\n"
952     f"{model_path}"
953 )
954
955
956 # =====
957 # 25. REALISTIC SAMPLE PREDICTION
958 # =====
959
960 print("\n" + "=" * 80)
961 print("SAMPLE STUDENT PREDICTION")
962 print("=" * 80)
963
964 sample_student = pd.DataFrame(
965     [
966         {
967             "gender": "female",
968             "race/ethnicity": "group D",
969             "parental level of education": "some college",
970             "lunch": "standard",
971             "test preparation course": "completed",
972         }
973     ]
974 )
975
976 sample_prediction = best_model.predict(
977     sample_student
978 )[0]
979
980 print("\nStudent information:")
981
982 print(
```

```
983     sample_student.to_string(index=False)
984 )
985
986 print(
987     f"\nPredicted overall score: "
988     f"{sample_prediction:.2f}"
989 )
990
991 print(
992     f"Performance category: "
993     f"{performance_category(sample_prediction)}"
994 )
995
996
997 # =====
998 # 26. SAVE PROJECT SUMMARY
999 # =====
1000
1001 summary = f"""
1002 STUDENT PERFORMANCE PREDICTION PROJECT
1003 =====
1004
1005 Dataset
1006 -----
1007 File: exams.csv
1008 Rows after cleaning: {len(df)}
1009
1010 Target
1011 -----
1012 overall_score
1013
1014 The target is calculated as the average of:
1015 - math score
1016 - reading score
1017 - writing score
1018
1019 Input Features
1020 -----
1021 {chr(10).join('- ' + feature for feature in feature_columns)}
1022
1023 Models
1024 -----
1025 1. Linear Regression
1026 2. Random Forest Regressor
1027
1028 Selected Model
1029 -----
```

```
1030 {best_model_name}
1031
1032 Best Model Metrics
1033 -----
1034 MAE : {results.loc[best_index, 'MAE']:.4f}
1035 RMSE : {results.loc[best_index, 'RMSE']:.4f}
1036 R2 : {results.loc[best_index, 'R2']:.4f}
1037
1038 Important Limitation
1039 -----
1040 The dataset does not contain:
1041 - study hours
1042 - attendance
1043 - previous grades
1044 - longitudinal student history
1045
1046 Therefore, those factors are not used.
1047
1048 Target leakage prevention
1049 -----
1050 Math, reading, and writing scores are not used as
1051 model input features because they directly determine
1052 the overall_score target.
1053
1054 Generated Outputs
1055 -----
1056 - cleaned_student_data.csv
1057 - model_comparison.csv
1058 - test_predictions.csv
1059 - feature_importance.csv
1060 - student_performance_model.pkl
1061 - overall_score_distribution.png
1062 - average_subject_scores.png
1063 - score_by_test_preparation.png
1064 - score_by_gender.png
1065 - score_by_parental_education.png
1066 - score_by_lunch.png
1067 - actual_vs_predicted.png
1068 """
1069
1070 summary_path = OUTPUT_DIR / "project_summary.txt"
1071
1072 with open(
1073     summary_path,
1074     "w",
1075     encoding="utf-8"
1076 ) as file:
```

```
1077     file.write(summary)
1078
1079 print(
1080     f"\nProject summary saved to:\n"
1081     f"{summary_path}"
1082 )
1083
1084
1085 # =====
1086 # 27. FINAL MESSAGE
1087 # =====
1088
1089 print("\n" + "=" * 80)
1090 print("WEEK 1 STUDENT PERFORMANCE PROJECT COMPLETED")
1091 print("=" * 80)
1092
1093 print("""
1094 The complete pipeline has finished.
1095
1096 Next:
1097 1. Review the printed EDA results.
1098 2. Review model_comparison.csv.
1099 3. Review the generated charts in outputs/.
1100 4. Check the sample prediction.
1101 5. Document the findings in README.md.
1102 """)
```

Listing 1: Complete Student Performance Prediction Python Program

19 Code Explanation by Major Section

19.1 Imports

The program imports libraries for:

- File path handling using `pathlib`
- Numerical operations using NumPy
- Data manipulation using Pandas
- Visualization using Matplotlib
- Machine Learning using Scikit-learn
- Model saving using Joblib

19.2 Paths

The program identifies its own directory using:

```
Path(__file__).resolve().parent
```

and constructs the dataset and output locations relative to the project directory.

19.3 Dataset Loading

The dataset is loaded using:

```
pd.read_csv(DATA_PATH)
```

which creates a Pandas DataFrame named `df`.

19.4 Validation

The implementation validates that the required dataset columns are present before continuing.

This prevents silent failure when a dataset has an unexpected schema.

19.5 Cleaning

The cleaning stage:

- Checks missing values.
- Checks duplicate rows.
- Checks score ranges.
- Converts scores to numeric values.
- Replaces invalid scores with `NaN`.
- Removes records where all three target-generating scores are missing.

19.6 Feature Engineering

A new feature called `overall_score` is created as the arithmetic mean of the three examination scores.

A second derived column called `performance_category` is created for interpretation.

19.7 Exploratory Data Analysis

The program examines:

- Summary statistics
- Category frequencies
- Group means
- Median scores
- Score distributions

19.8 Preprocessing

Categorical variables are processed using an imputer and one-hot encoder.

The `Pipeline` ensures that preprocessing and prediction remain consistent.

19.9 Training

The program trains:

1. Linear Regression
2. Random Forest Regression

19.10 Evaluation

Each model is evaluated using:

- MAE
- RMSE
- R^2

19.11 Model Selection

The model with the smallest RMSE is selected automatically.

19.12 Prediction

The selected model predicts scores for unseen test examples and for an example student profile.

19.13 Serialization

The trained model pipeline is saved as a `.pkl` file using Joblib.

20 Possible Interview Questions

1. Why is this problem formulated as regression?
2. What is the target variable?
3. What are the feature variables?
4. Why are math, reading, and writing scores excluded from the input features?
5. What is target leakage?
6. Why do categorical features need encoding?
7. What does OneHotEncoder do?
8. Why is a Pipeline useful?
9. Why do we split the data into training and test sets?
10. What is the purpose of `random_state=42`?
11. Why was Linear Regression selected as a baseline?

12. Why was Random Forest tested?
13. What does MAE measure?
14. What does RMSE measure?
15. What does R^2 measure?
16. Why can RMSE be useful when evaluating score predictions?
17. What does `n_estimators=300` mean?
18. What does `n_jobs=-1` mean?
19. What is feature importance?
20. Does feature importance imply causation?
21. What limitations does the dataset have?
22. What additional features would make the system more useful?

21 Future Improvements

Potential improvements include:

- Adding attendance information
- Adding study-hour information
- Adding previous grades
- Adding assignment performance
- Adding longitudinal student history
- Testing additional regression algorithms
- Using cross-validation
- Performing hyperparameter tuning
- Building a Streamlit user interface
- Adding automated model monitoring
- Deploying the application

22 Conclusion

This project demonstrates a complete introductory Machine Learning workflow rather than only training a model.

The implementation starts with raw data and performs:

**Validation → Cleaning → Exploration → Feature Engineering →
Preprocessing → Training → Evaluation → Prediction**

The project also demonstrates an important AI engineering principle: the quality of a Machine Learning solution depends not only on the algorithm but also on correct problem formulation, data preparation, evaluation, and understanding of limitations.